Below is a **version-wise view of how SOLID principles are reflected in java.lang.String from Java 1.0 to Java 21**.

Note: String is final, immutable, and part of java.lang since Java 1.0.
SOLID is an OO design guideline (not explicitly declared in JDK), but its principles are clearly reflected in the design of String.

SOLID Principle Implementation in String (Java 1 → 21)

Java Version	S – Single Responsibility	O – Open/Closed	L – Liskov Substitution	I – Interface Segregation	D – Dependency Inversion
Java 1.0	Represents immutable text only	final class (cannot be modified)	Can be used wherever Object expected	Minimal interface exposure	Depends on abstractions (Object)
Java 1.4	Regex logic delegated to Pattern	Added replaceAll(), split() without modifying behavior	Still safe substitution	Regex separated via java.util.regex	Depends on regex abstraction
Java 5	Formatting delegated to Formatter	format() added without breaking old API	Works polymorphically	CharSequence introduced	Uses CharSequence abstraction
Java 6	isEmpty() added (text responsibility only)	API extended safely	No behavioral change	Still lean API	No tight coupling
Java 7	No responsibility change	switch(String) support at compiler level	Substitution unchanged	No extra burden	Compiler handles logic
Java 8	join() added	Static utility extension	Compatible everywhere	Uses Iterable abstraction	Depends on Iterable interface
Java 9	Internal storage changed (char[] → byte[])	Closed for modification but open for optimization	Behavior unchanged	Stream methods added	Returns IntStream abstraction

Java 11	Text utilities (strip, repeat, lines)	Extended safely	No contract break	Stream-based APIs	Uses Stream abstraction
Java 12–15	Text blocks support	Added indent(), transform(), formatted()	Substitution preserved	Functional interfaces used	Depends on Function abstraction
Java 16–21	Performance refinements only	No breaking changes	Fully substitutable	No unnecessary interfaces added	Uses standard abstractions only

---

□ Detailed SOLID Analysis for String

Principle	How String Implements It
S – Single Responsibility	Only responsible for representing immutable text. Formatting, regex, streams, encoding handled by separate classes.
O – Open/Closed	final class prevents modification. Extended through new methods without breaking old code.
L – Liskov Substitution	String can be used anywhere Object, CharSequence, Serializable, Comparable<String> is expected.
I – Interface Segregation	Implements only necessary interfaces: CharSequence, Comparable, Serializable, Constable, ConstantDesc (Java 12+).
D – Dependency Inversion	Depends on abstractions like CharSequence, Iterable, Function, Stream, not concrete implementations.

□ Interfaces Implemented Over Time

Java Version	Interfaces Implemented
Java 1.0	Serializable, Comparable<String>
Java 5	CharSequence
Java 12	Constable, ConstantDesc

□ Key Design Decisions Showing SOLID

1. **Immutability (SRP + OCP)**
   - No setter methods
   - Thread-safe
   - Predictable behavior
2. **Final Class (OCP)**
   - Prevents subclass misuse
   - Maintains security & consistency
3. **Use of Interfaces (ISP + DIP)**
   - CharSequence
   - Iterable
   - Function
   - Stream
4. **Internal Optimization Without Breaking API (OCP)**
   - Java 9 Compact Strings
   - Java 18 UTF-8 default

□ Final Conclusion

From **Java 1 to Java 21**, the String class consistently follows SOLID principles:

- Strongest principles applied: **SRP, OCP, LSP**
- Carefully designed interface implementation (ISP)
- Heavy use of abstractions (DIP)
- Backward compatibility maintained for 25+ years

If you want next, I can provide:

- ✅ SOLID violation examples (if String were badly designed)
- ✅ SOLID comparison: String vs StringBuilder
- ✅ Interview-ready explanation (2-minute answer)
- ✅ UML-style design diagram explanation

Below is a **comprehensive table of all major software design principles reflected in java.lang.String from Java 1.0 to Java 21**.

Note: String has remained immutable, final, and backward compatible since Java 1.0.
The principles listed are design principles reflected in its architecture and evolution.

□ Design Principles Implemented in java.lang.String (Java 1 → 21)

#	Design Principle	Implemented Since	How It Is Reflected in String
1	**Immutability Principle**	Java 1.0	String objects cannot be modified after creation
2	**Single Responsibility Principle (SRP)**	Java 1.0	Only represents immutable character sequences
3	**Open/Closed Principle (OCP)**	Java 1.0	final class, extended via new methods without breaking old behavior
4	**Liskov Substitution Principle (LSP)**	Java 1.0	Works wherever Object, CharSequence, Comparable expected
5	**Interface Segregation Principle (ISP)**	Java 5	Implements minimal required interfaces (CharSequence, etc.)
6	**Dependency Inversion Principle (DIP)**	Java 5+	Uses abstractions like CharSequence, Function, Stream
7	**Encapsulation**	Java 1.0	Internal array (char[] → byte[]) hidden from users
8	**Information Hiding**	Java 1.0	Private internal storage not directly accessible
9	**Final Class Security Principle**	Java 1.0	Prevents subclass-based security issues
10	**Fail-Fast Principle**	Java 1.0	Throws IndexOutOfBoundsException, NullPointerException appropriately

11	**Defensive Copying**	Java 1.0	Returns new objects instead of exposing internal array
12	**Value Object Design Pattern**	Java 1.0	Represents pure value, equality based on content
13	**Flyweight Pattern (String Pool)**	Java 1.0	intern() and string literal pooling
14	**Factory Method Pattern**	Java 1.0	valueOf(), format() static creation methods
15	**Strategy Pattern (Regex)**	Java 1.4	Delegates pattern matching to Pattern class
16	**Separation of Concerns**	Java 1.4+	Regex, formatting, streams handled externally
17	**Functional Programming Principle**	Java 8+	transform(), formatted(), stream methods
18	**Stream API Integration**	Java 9	chars(), codePoints(), lines()
19	**Backward Compatibility Principle**	Java 1 → 21	No breaking API changes
20	**Performance Optimization Principle**	Java 9	Compact Strings (char[] → byte[])
21	**Unicode Compliance Principle**	Java 1.0	UTF-16 support, codePoint methods
22	**Thread-Safety by Design**	Java 1.0	Immutable → inherently thread-safe
23	**Low Coupling Principle**	Java 5+	Depends on abstractions not implementations
24	**High Cohesion Principle**	Java 1.0	All methods tightly related to string manipulation
25	**Least Surprise Principle**	Java 1.0	Predictable behavior for equality and operations
26	**DRY (Don't Repeat Yourself)**	Java 8+	Utility methods reduce repetitive code (join, repeat)
27	**Fluent API Support**	Java 11+	Methods return new String allowing chaining

28	**Memory Efficiency Principle**	Java 9	Compact Strings optimization
29	**Locale Sensitivity Principle**	Java 1.1+	toLowerCase(Locale), format(Locale, ...)
30	**Constant API Stability Principle**	Java 1 → 21	API stable for 25+ years

---

□ Interfaces Supporting These Principles

Java Version	Interfaces Implemented
Java 1.0	Serializable, Comparable<String>
Java 5	CharSequence
Java 12	Constable, ConstantDesc

□ Summary

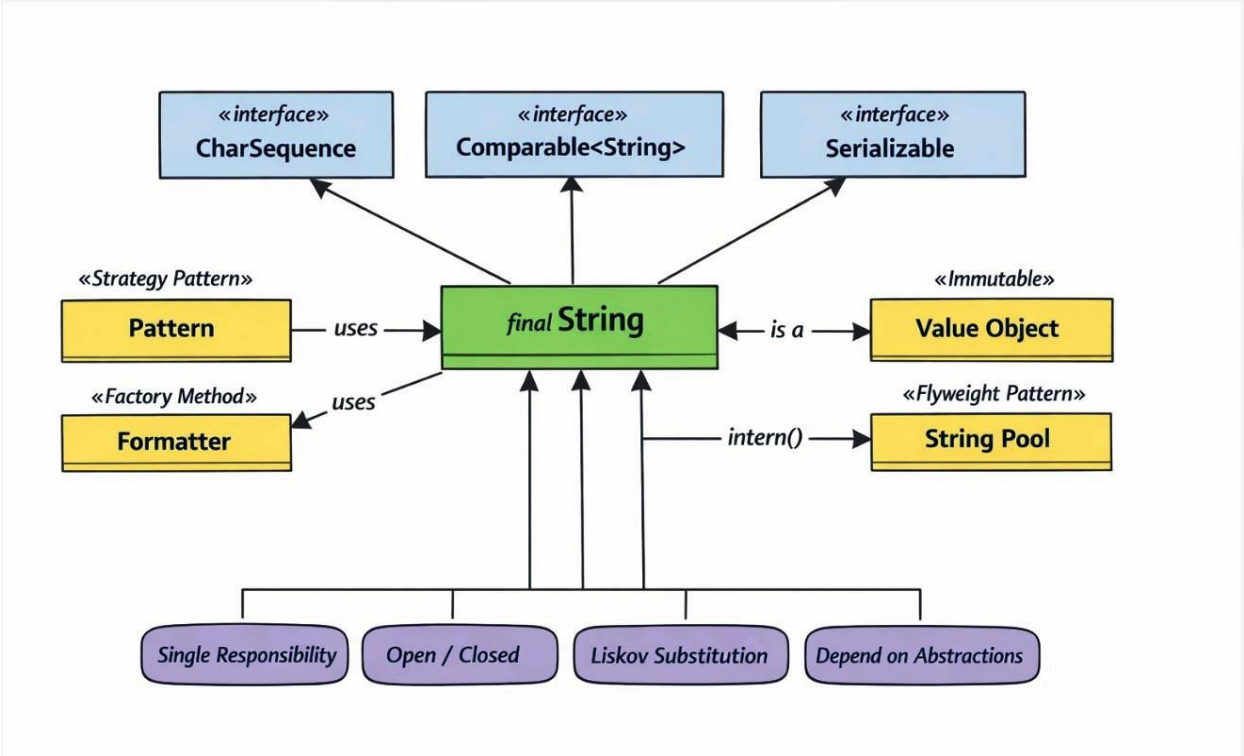
From **Java 1 to Java 21**, String demonstrates:

- Core OO principles (SOLID)
- Design patterns (Flyweight, Factory, Value Object)
- Functional programming integration
- Backward compatibility for 25+ years
- Internal optimization without breaking API

It is considered one of the best-designed core classes in the JDK.

If you'd like next, I can provide:

- ✅ UML-style architectural explanation
- ✅ Comparison with StringBuilder and StringBuffer
- ✅ Interview-ready summary
- ✅ Printable revision notes
- ✅ Internal memory structure evolution table



Here is the **UML-style architectural explanation of `java.lang.String` (Java 1 → 21)** in structured diagram format.

□ UML Architectural View of String (Java 21)

```
+-------------------------------------+
|          <<final>>                  |
|           String                    |
+-------------------------------------+
| - value : byte[]                    |  (char[] until Java 8)
| - coder : byte                      |  (Java 9+ Compact String)
+-------------------------------------+
| + length() : int                    |
| + charAt(int) : char                |
| + substring(int,int) : String       |
| + equals(Object) : boolean          |
| + compareTo(String) : int           |
| + isBlank() : boolean               |
| + strip() : String                  |
| + repeat(int) : String              |
| + lines() : Stream<String>          |
| + format(...) : static String       |
| + join(...) : static String         |
| + intern() : String                 |
+-------------------------------------+
```

□ Implemented Interfaces (UML Representation)

```
+-----------------------+
| CharSequence     |
+-----------------------+
        ▲
        |
+---------------------------+
| Comparable<String>  |
+---------------------------+
        ▲
        |
+--------------------+
| Serializable      |
+--------------------+
        ▲
        |
+--------------------+
|   Constable       |  (Java 12+)
+--------------------+
        ▲
        |
+--------------------+
| ConstantDesc   |  (Java 12+)
+--------------------+
        ▲
        |
    +--------+
    | String |
    +--------+
```

□ Internal Collaboration (Dependency UML)

```
String
  │
  ├── uses → Pattern           (regex operations)
  ├── uses → Formatter         (format())
  ├── returns → Stream         (lines())
  ├── returns → IntStream      (chars(), codePoints())
  ├── uses → Locale            (case conversion)
  └── interacts → String Pool (intern())
```

□ Design Pattern Representation (Conceptual UML)

Pattern	UML Interpretation	Purpose
Value Object	Immutable, equality by content	Safe & thread-safe
Flyweight	String Pool	Memory optimization
Factory Method	valueOf(), format()	Controlled object creation
Strategy	Delegates regex to Pattern	Separation of concerns

□ SOLID Reflection in Architecture

Principle	UML Evidence
SRP	Only text representation responsibility
OCP	final class + API extension without breaking
LSP	Substitutable for Object, CharSequence
ISP	Implements minimal interfaces
DIP	Uses abstractions (Stream, Function, Iterable)

□ Evolution Diagram (Storage Change)

```
Java 1 – 8:
String
  └── char[] value

Java 9 – 21:
String
  ├── byte[] value
  └── byte coder   (LATIN1 or UTF16)
```

✔ API unchanged

✔ Performance improved

✔ Memory optimized

□ Final Architectural Summary

String is:

- final → prevents inheritance misuse
- Immutable → thread-safe
- High cohesion, low coupling
- Stable API for 25+ years
- Optimized internally without breaking contracts

If you want, I can generate:

- □ Professional UML class diagram (clean exam-ready format)
- □ PNG image version (downloadable)
- □ Comparison UML: String vs StringBuilder
- □ Interview explanation (2-minute architecture answer)

Below is a **version-wise comparison of all String methods added from Java 1.0 to Java 21**.

Notes:
• Only methods defined in java.lang.String are included (not inherited from Object).
• Methods are grouped by the Java version in which they were introduced.
• If a version had no new String methods, it is marked accordingly.

□ Complete Version-Wise String Methods (Java 1 → 21)

Java Version	Methods Introduced in That Version
Java 1.0	length(), isEmpty() (added later officially Java 6), charAt(int), getChars(), getBytes(), equals(), equalsIgnoreCase(), compareTo(), compareToIgnoreCase(), regionMatches(), startsWith(), endsWith(), hashCode(), indexOf(), lastIndexOf(), substring(), concat(), replace(char,char), toLowerCase(), toUpperCase(), trim(), toString(), toCharArray(), valueOf(), copyValueOf(), intern()
Java 1.1	No new public methods
Java 1.2	No new public methods
Java 1.3	No new public methods
Java 1.4	matches(), replaceFirst(), replaceAll(), split()

Java 5 (1.5)	format(), format(Locale, ...), contentEquals(CharSequence), contentEquals(StringBuffer)
Java 6	isEmpty()
Java 7	No new methods (but switch support added)
Java 8	join() (static), join(CharSequence, Iterable)
Java 9	chars(), codePoints()
Java 10	No new methods
Java 11	isBlank(), lines(), strip(), stripLeading(), stripTrailing(), repeat()
Java 12	indent(), transform()
Java 13	stripIndent() (related to Text Blocks)
Java 14	No new methods
Java 15	formatted()
Java 16	No new methods
Java 17 (LTS)	No new methods
Java 18	No new methods (UTF-8 default charset change)
Java 19	No new methods
Java 20	No new methods
Java 21 (LTS)	No new public String methods

□ Summary of Major Method Additions

Feature Type	Methods	Java Version
Regex Support	matches(), replaceAll(), replaceFirst(), split()	Java 1.4
Formatting	format(), formatted()	Java 5, Java 15
Utility Checks	isEmpty(), isBlank()	Java 6, Java 11

Stream Support	chars(), codePoints()	Java 9
Joining	join()	Java 8
Modern Text Handling	strip*(), repeat(), lines()	Java 11
Functional Style	transform()	Java 12
Indentation	indent(), stripIndent()	Java 12–13

 Important Clarification

From **Java 16 → 21**, no new public methods were added to String.
Most improvements were:

- Performance optimizations
- JVM internal changes
- Memory efficiency improvements
- Encoding default change (Java 18)

If you want next, I can provide:

- ✅ Full categorized list of all 70+ current String methods
- ✅ Java 8 vs 11 vs 17 vs 21 comparison table
- ✅ Interview-oriented String evolution summary
- ✅ Internal implementation changes (char[] → byte[])
- ✅ Printable PDF-style structured notes

Below is the **complete categorized list of all current public methods of java.lang.String (Java 21 LTS)**.

✔ Includes instance + static methods

✔ Overloaded methods grouped together

✔ Excludes methods inherited from Object

✔ Based on Java 21 API

---

java.lang.String (Java 21) – Complete Categorized Method List

---

1 Basic Information Methods

Method
int length()
boolean isEmpty()
boolean isBlank()
int hashCode()

---

2 Character Access Methods

Method
char charAt(int index)
int codePointAt(int index)
int codePointBefore(int index)
int codePointCount(int beginIndex, int endIndex)
int offsetByCodePoints(int index, int codePointOffset)

---

3 Comparison Methods

Method
boolean equals(Object obj)
boolean equalsIgnoreCase(String anotherString)
int compareTo(String anotherString)
int compareToIgnoreCase(String str)
boolean contentEquals(StringBuffer sb)

boolean contentEquals(CharSequence cs)
boolean regionMatches(int toffset, String other, int ooffset, int len)
boolean regionMatches(boolean ignoreCase, int toffset, String other, int ooffset, int len)

---

4 Searching Methods

Method
boolean contains(CharSequence s)
int indexOf(int ch)
int indexOf(int ch, int fromIndex)
int indexOf(String str)
int indexOf(String str, int fromIndex)
int lastIndexOf(int ch)
int lastIndexOf(int ch, int fromIndex)
int lastIndexOf(String str)
int lastIndexOf(String str, int fromIndex)
boolean startsWith(String prefix)
boolean startsWith(String prefix, int toffset)
boolean endsWith(String suffix)

---

5 Substring Methods

Method
String substring(int beginIndex)
String substring(int beginIndex, int endIndex)
CharSequence subSequence(int beginIndex, int endIndex)

---

6 Concatenation & Joining

Method
String concat(String str)
static String join(CharSequence delimiter, CharSequence... elements)
static String join(CharSequence delimiter, Iterable<? extends CharSequence> elements)

---

7 Replacement Methods

Method
String replace(char oldChar, char newChar)
String replace(CharSequence target, CharSequence replacement)
String replaceAll(String regex, String replacement)
String replaceFirst(String regex, String replacement)

---

8 Case Conversion

Method
String toLowerCase()
String toLowerCase(Locale locale)
String toUpperCase()
String toUpperCase(Locale locale)

---

9 Whitespace Handling

Method
String trim()
String strip()
String stripLeading()
String stripTrailing()

String indent(int n)
String stripIndent()

---

10 Splitting & Lines

Method
String[] split(String regex)
String[] split(String regex, int limit)
Stream<String> lines()

---

11 Formatting Methods

Method
static String format(String format, Object... args)
static String format(Locale locale, String format, Object... args)
String formatted(Object... args)

---

12 Repetition & Transformation

Method
String repeat(int count)
<R> R transform(Function<? super String, ? extends R> f)

---

13 Conversion Methods

Method
char[] toCharArray()
byte[] getBytes()
byte[] getBytes(Charset charset)
byte[] getBytes(String charsetName)
void getChars(int srcBegin, int srcEnd, char[] dst, int dstBegin)

String toString()

---

14 Static Value Conversion Methods

Method
static String valueOf(boolean b)
static String valueOf(char c)
static String valueOf(char[] data)
static String valueOf(char[] data, int offset, int count)
static String valueOf(int i)
static String valueOf(long l)
static String valueOf(float f)
static String valueOf(double d)
static String valueOf(Object obj)
static String copyValueOf(char[] data)
static String copyValueOf(char[] data, int offset, int count)

---

15 Unicode Stream Methods (Java 9+)

Method
IntStream chars()
IntStream codePoints()

---

16 Pattern Matching

Method
boolean matches(String regex)

---

17 Interning

Method
String intern()

✅ Total Public Methods (Java 21)

✔ **70+ public methods (including overloaded variants)**

✔ No new public methods added after Java 15

✔ Most recent additions: formatted(), stripIndent(), indent(), transform()

If you'd like next, I can provide:

- □ Interview-focused cheat sheet
- □ Printable exam revision table
- □ Java 8 vs 11 vs 17 vs 21 difference table
- □ All String methods with examples
- □ Internal memory architecture (Java 8 vs 9 comparison)